DOCSAFE EVIDENCE CHAIN PROTOCOL

Complete Algorithms & Technical System Specification

A Comprehensive Formal Exposition of Cryptographic Hashing, Dual-Party Consensus, Algorithm 2 Dispute Resolution, Decentralized IPFS Storage, Role-Based Access Control, and Tamper-Evident Court-Admissible Chain-of-Custody

Protocol: DocSafe Ethereum Engine v2.0	**Target Network:** Ethereum EVM / Hardhat 31337	**Storage:** IPFS + SQLite Embedded
Smart Contract: DocumentVerification.sol	**Crypto Primitives:** SHA-256, ECDSA secp256k1	**Standard:** Court-Admissible Evidence Chain

---

1. System Architecture & The 4-Layer Mental Model

The DocSafe Evidence Chain system solves the fundamental challenges of digital forensic evidence management: unauthorized tampering, broken chains of custody, unilateral overwrites, opaque handling, and the excessive cost of raw blockchain storage. Rather than storing large evidentiary files (such as high-resolution autopsy photographs, forensic laboratory reports, or surveillance CCTV footage) directly on-chain, DocSafe operates on a **four-layer hybrid architecture**:

Layer	Components & Technology	Forensic & Cryptographic Responsibilities
Layer 1: Presentation & Web3 Client	React 19, Vite, Wagmi v2, Viem v2, MetaMask EIP-1193	User interaction, local SHA-256 pre-hashing, transaction signature creation, role routing, visual timeline rendering, and court certificate generation.
Layer 2: Gateway & Metadata Engine	Node.js 24, Express, Native SQLite DatabaseSync, Pinata SDK	Off-chain case metadata persistence (case numbers, forensic categories, notes), server-side SHA-256 verification, IPFS pinning orchestration, proxy streaming with integrity headers.
Layer 3: Decentralized Storage	InterPlanetary File System (IPFS), UnixFS, Content Addressing	Permanent, distributed off-chain storage. Files are addressed strictly by their cryptographic Content Identifier (CID). Any mutation results in a distinct CID.
Layer 4: Consensus & Ledger	Ethereum EVM, Solidity 0.8.0+, DocumentVerification Contract	Immutable state machine enforcement: dual-party consent gates, multi-validator dispute arbitration (Algorithm 2), viewer access control, and tamper-proof event logs.

2. Algorithm 1: Evidence Creation & Dual-Party Authorization Protocol

Purpose: Enforces a strict two-stage ingestion protocol preventing unauthorized, unsolicited, or unvetted evidence from polluting the evidentiary chain. A provider cannot unilaterally upload files; an empty container must first be initialized and explicitly consented to by the designated recipient.

ALGORITHM 1: EVIDENCE CREATION & DUAL-PARTY AUTHORIZATION PROTOCOL **Time: O(1) | Gas: ~85,000 to ~120,000 gas**

Inputs: recipientAddress, caseMetadata, evidenceFile
Outputs: docId, IPFS CID, on-chain DocumentAdded and DocumentUpdated events

```
Algorithm 1: Evidence Container Creation and Dual-Party Consent Gate
1: procedure ADD_EMPTY_DOCUMENT(recipient_addr)
2: require msg.sender != recipient_addr and recipient_addr != 0x0
3: docId ← documentCount + 1
4: documents[docId] ← Document(ipfsHash="", provider=msg.sender, recipient=recipient_addr,
5: isValid=false, isInChangePhase=false, proof="", sha256="", isRejected=false)
6: viewers[msg.sender][docId] ← true; viewers[recipient_addr][docId] ← true
7: emit DocumentAdded(docId, msg.sender, recipient_addr, "")
8: return docId
9: end procedure
10:
11: procedure APPROVE_DOCUMENT_CHANGE(docId)
12: require msg.sender == documents[docId].recipient
13: require documents[docId].isRejected == false
14: approvals[msg.sender][docId] ← true
15: end procedure
16:
17: procedure UPDATE_DOCUMENT(docId, ipfsCid, sha256Hex)
18: require msg.sender == documents[docId].provider
19: require approvals[documents[docId].recipient][docId] == true
20: require documents[docId].isRejected == false
21: documents[docId].ipfsHash ← ipfsCid
22: documents[docId].sha256Hash ← sha256Hex
23: documents[docId].isValid ← false
24: approvals[documents[docId].recipient][docId] ← false // Consumes one-time authorization token
25: emit DocumentUpdated(docId, ipfsCid)
26: end procedure
```

Technical Analysis & Invariants:

- **Two-Phase Handshake:** In Stage 1, an empty container is indexed. In Stage 2, the recipient inspects the case context (Case Number, Evidence Title, Jurisdiction) and issues an authorization token via approveDocumentChange. Only then can the provider's file upload transaction succeed.
- **One-Time Authorization Token Consumption:** Line 24 explicitly resets approvals[recipient][docId] = false. This ensures that once an evidence file is registered, the provider cannot perform silent subsequent overwrites without obtaining fresh consent.
- **Access Shielding:** Upon container initialization, viewers[msg.sender][docId] and viewers[recipient][docId] are granted atomic viewing capabilities while the rest of the world is blocked.

3. Algorithm 2: Dispute Resolution, Force Update & Challenge Consensus

Purpose: Resolves post-validation evidentiary revisions and adversarial disputes. In real-world legal proceedings, forensic evidence may require updates (e.g. revised toxicological analysis or supplemental ballistics findings). Algorithm 2 prevents unilateral tampering by either party through an adversarial challenge mechanism arbitrated by an independent quorum of authorized validators.

ALGORITHM 2: FORCE UPDATE, ADVERSARIAL CHALLENGE & DISPUTE ARBITRATION **Time: O(1) per vote | Gas: ~45,000 to ~95,000 gas per tx**

Inputs: docId, revisedIpfsHash, proofString, witnessStatement, validatorSignature
Outputs: Dispute round increment, updated isValid state, DocumentChallenge and DocumentValidated events

```
Algorithm 2: Adversarial Challenge, Multi-Validator Arbitration & Anti-Deadlock Quorum
1: procedure FORCE_UPDATE_DOCUMENT(docId, revisedIpfsHash, proofText)
2: require msg.sender == documents[docId].provider and !documents[docId].isRejected
3: if documents[docId].isValid == true then
4: validDocuments[docId] ← documents[docId] // Preserves prior verified state
5: end if
6: disputeRounds[docId] ← disputeRounds[docId] + 1 // Increment round to reset vote map
7: validations[docId] ← 0; documents[docId].isValid ← false
8: documents[docId].isInChangePhase ← true; documents[docId].proof ← proofText
9: documents[docId].ipfsHash ← revisedIpfsHash
10: emit DocumentChallenge(docId, revisedIpfsHash, proofText, "")
11: end procedure
12:
13: procedure CHALLENGE_DOCUMENT(docId, counterHash, witnessStatement)
14: require msg.sender == documents[docId].recipient and !documents[docId].isRejected
15: require documents[docId].isInChangePhase == true
16: documents[docId].ipfsHash ← counterHash; documents[docId].isInChangePhase ← false
17: emit DocumentChallenge(docId, counterHash, documents[docId].proof, witnessStatement)
18: end procedure
19:
20: procedure VALIDATE_FROM_VALIDATORS(docId)
21: require validators[msg.sender] == true and !documents[docId].isRejected
22: r ← disputeRounds[docId]
23: require hasValidatedRound[docId][r][msg.sender] == false // Prevents double-voting in round
24: hasValidatedRound[docId][r][msg.sender] ← true; validations[docId] ← validations[docId] + 1
25: if validations[docId] ≥ validationLimit then
26: documents[docId].isValid ← true; documents[docId].isInChangePhase ← false
27: validDocuments[docId] ← documents[docId]
28: emit DocumentValidated(docId, msg.sender)
29: end if
30: end procedure
```

Mathematical Model of Dispute Rounds & Anti-Deadlock Guarantee:

Let $R \in \mathbb{N}$ represent the dispute round sequence. In traditional multi-sig designs, if a validator votes in round r, an internal boolean mapping hasValidated[val] = true permanently locks that validator from voting again. If a case is contested multiple times, the system enters an unrecoverable deadlock. DocSafe overcomes this by parameterizing votes over a 3-dimensional tensor: $\texttt{hasValidatedRound} : \text{DocID} \times \text{Round} \times \text{Validator} \to \{0, 1\}$. When a new dispute round is initiated ($r' = r + 1$), the previous round's votes are sealed in the historical audit trail, and a fresh consensus quorum threshold $k = \text{validationLimit}$ is established with zero gas cleanup cost.

4. Algorithm 3: MetaMask Web3 Cryptographic Signing & IPFS Binding

Purpose: Manages off-chain cryptographic digital signature generation via MetaMask and content-addressed storage on IPFS. Ensures non-repudiation of transactions while minimizing on-chain gas costs.

ALGORITHM 3: METAMASK WEB3 TRANSACTION SIGNING & IPFS OFF-CHAIN BINDING **Time: O(N) file hash + O(1) tx | Gas: ~55,000 gas**

Inputs: fileBytes, docId, providerAddress, MetaMask Wallet
Outputs: IPFS Content Identifier (CID), on-chain transaction receipt, immutable ledger anchor

```
Algorithm 3: Client-Side ECDSA Signing and Off-Chain Decentralized Storage Binding
procedure INGEST_AND_BIND_EVIDENCE(fileBytes, docId, signerAccount)
H_local ← SHA256(fileBytes) // Client-side WebCrypto digest
payload ← { file: fileBytes, docId: docId, provider: signerAccount }
uploadRes ← POST /upload(payload)
require uploadRes.sha256 == H_local // Pre-upload integrity match
CID ← uploadRes.ipfsHash
txData ← encodeABI("updateDocument", [docId, CID, H_local])
txParams ← { from: signerAccount, to: CONTRACT_ADDRESS, data: txData }
txHash ← eth_sendTransaction(txParams) via MetaMask EIP-1193
receipt ← waitForTransactionReceipt(txHash)
emit OffChainRegistered(docId, CID, H_local, receipt.blockNumber)
end procedure
```

Cryptographic Primitive Rationale:

- **Elliptic Curve Digital Signature Algorithm (ECDSA):** MetaMask signs transaction payloads with the secp256k1 curve ($y^2 = x^3 + 7 \pmod p$). The transaction hash e and private key d yield signature (r, s, v). On-chain EVM executes ecrecover to cryptographically guarantee that the provider address matches msg.sender.
- **Storage Cost Optimization:** Storing 1 MB of binary evidence directly on Ethereum at 20 gwei gas price costs upwards of $1,500. Storing the IPFS CID (32 to 64 bytes) alongside the 32-byte SHA-256 fingerprint costs under $0.05 while achieving equivalent mathematical immutability.

5. Algorithm 4: Cryptographic Pre/Post Integrity Verification

Purpose: Implements zero-knowledge end-to-end evidence integrity auditing. A common vulnerability in naive blockchain systems is the 'Garbage-In, Garbage-Out' flaw, where an attacker alters file bytes during gateway transit or IPFS retrieval. Algorithm 4 guarantees dual-layer integrity verification at every hop of the evidentiary lifecycle.

ALGORITHM 4: CRYPTOGRAPHIC PRE/POST INTEGRITY VERIFICATION & STREAMING DIGEST **Time: O(N) streaming linear | Gas: Off-chain ($0 gas)**

Inputs: fileBuffer, expectedSha256, IPFS CID
Outputs: Integrity confirmation boolean, X-SHA256-Fingerprint HTTP header

```
Algorithm 4: Dual-Digest Verification (Client SHA-256 + Gateway Proxy Stream Integrity)
1: procedure VERIFY_EVIDENCE_INTEGRITY(fileBuffer, expectedSha256)
2: computedDigest ← crypto.subtle.digest("SHA-256", fileBuffer)
3: computedHex ← toHexString(computedDigest)
4: if computedHex != expectedSha256 then
5: raise IntegrityViolationError("Cryptographic hash mismatch! Evidence may be tampered.")
6: end if
7: return { verified: true, digest: computedHex }
8: end procedure
9:
10: procedure GATEWAY_PROXY_FETCH(cid)
11: rawStream ← GET https://gateway.pinata.cloud/ipfs/cid
12: hasher ← createHash("sha256"); chunks ← []
13: for each chunk in rawStream do: hasher.update(chunk); chunks.push(chunk)
14: streamSha256 ← hasher.digest("hex")
15: setResponseHeader("X-SHA256-Fingerprint", streamSha256)
16: returnStream(chunks)
17: end procedure
```

Forensic Admissibility Guarantee (Federal Rule of Evidence 902(13)/(14)): *By comparing the raw binary hash generated at ingest against the on-chain recorded fingerprint and the gateway stream digest, DocSafe satisfies the legal standard for Self-Authenticating Electronic Data.*

6. Algorithm 5: Evidence Rejection, Rollback & Linked Resubmission

Purpose: Implements formal evidentiary rejection and lineage-linked container resubmission. If submitted evidence is corrupted, illegible, or violates chain-of-custody protocols, reviewers reject it. Algorithm 5 closes the review round, freezes the defective container, and allows the original provider to spin up a linked replacement container without destroying historical audit evidence.

ALGORITHM 5: FORMAL REJECTION, STATE FREEZING & LINEAGE RESUBMISSION **Time: O(1) | Gas: ~35,000 to ~65,000 gas**

Inputs: docId, rejectionReason, previousDocId
Outputs: DocumentRejected event, DocumentResubmitted event, new linked container ID

```
Algorithm 5: Formal Rejection, State Machine Freezing and Lineage Resubmission
1: procedure REJECT_DOCUMENT(docId, reasonString)
2: require msg.sender == documents[docId].recipient or validators[msg.sender] or msg.sender == owner
3: require documents[docId].isValid == false // Cannot reject verified court evidence
4: require documents[docId].isRejected == false // Prevent redundant double-rejection
5: documents[docId].isRejected ← true
6: documents[docId].rejectionReason ← reasonString
7: documents[docId].isInChangePhase ← false
8: approvals[documents[docId].recipient][docId] ← false // Revoke consent
9: emit DocumentRejected(docId, msg.sender, reasonString)
10: end procedure
11:
12: procedure CREATE_RESUBMISSION(previousDocId, optionalRecipient)
13: require documents[previousDocId].provider == msg.sender
14: require documents[previousDocId].isRejected == true // Only rejected records can be resubmitted
15: targetRec ← (optionalRecipient != 0x0) ? optionalRecipient : documents[previousDocId].recipient
16: newDocId ← ADD_EMPTY_DOCUMENT(targetRec)
17: emit DocumentResubmitted(newDocId, previousDocId)
18: return newDocId
19: end procedure
```

7. Algorithm 6: Dynamic Role-Based Access Control & Viewer Delegation

Purpose: Implements dynamic on-chain capability delegation. Evidentiary records often need to be inspected by third-party expert witnesses, judges, or defense attorneys without granting them permission to alter the evidence or approve changes. Algorithm 6 provides granular viewing delegation under the strict authority of the case recipient.

ALGORITHM 6: DYNAMIC ROLE-BASED ACCESS CONTROL & VIEWER DELEGATION **Time: O(1) | Gas: ~28,000 gas per grant/revoke**

Inputs: docId, viewerAddress
Outputs: ViewerApproved / ViewerRevoked events, access-gated IPFS CID string

```
Algorithm 6: Recipient-Governed Dynamic Capability Delegation Matrix
1: procedure APPROVE_VIEWER(docId, viewerAddress)
2: require msg.sender == documents[docId].recipient and viewerAddress != 0x0
3: viewers[viewerAddress][docId] ← true
4: emit ViewerApproved(docId, viewerAddress)
5: end procedure
6:
7: procedure REVOKE_VIEWER(docId, viewerAddress)
8: require msg.sender == documents[docId].recipient
9: viewers[viewerAddress][docId] ← false
10: emit ViewerRevoked(docId, viewerAddress)
11: end procedure
12:
13: function VIEW_DOCUMENT(docId) view returns (string)
14: require viewers[msg.sender][docId] == true, "Access denied: Not authorized"
15: return documents[docId].ipfsHash
16: end function
```

8. Algorithm 7: Chain-of-Custody Timeline Aggregation & Event Decoding

Purpose: Aggregates on-chain decoded event logs and relational SQLite audit records into a single, unified, chronological timeline of custody. This algorithm powers both the interactive visual timeline and the printable forensic custody certificate.

ALGORITHM 7: CHAIN-OF-CUSTODY TIMELINE AGGREGATION & FORENSIC EVENT DECODING — **Time: O(L log L) sorting L logs | Gas: Off-chain ($0 gas)**

Inputs: docId, contractAddress, publicClient instance
Outputs: Chronologically unified audit timeline array, court-admissible certificate payload

```
Algorithm 7: Hybrid On-Chain/Off-Chain Chain-of-Custody Reconciliation
procedure FETCH_COMPLETE_AUDIT_TRAIL(docId, contractAddr)
meta ← GET /api/evidence/docId; backendEvents ← GET /api/evidence/docId/audit
rawLogs ← publicClient.getLogs({ address: contractAddr, fromBlock: 0n, toBlock: 'latest' })
timeline ← []
for each log in rawLogs do
decoded ← decodeEventLog({ abi: ABI, data: log.data, topics: log.topics })
if decoded.args.documentId == docId or decoded.args.newDocumentId == docId then
block ← publicClient.getBlock({ blockNumber: log.blockNumber })
timestamp ← formatISO(block.timestamp)
timeline.push(formatContractEvent(decoded.name, decoded.args, log.txHash, timestamp))
end if
end for
for each be in backendEvents do: timeline.push(formatBackendEvent(be))
timeline.sort(by timestamp ascending)
return { metadata: meta, timeline: timeline }
end procedure
```

9. Algorithmic Comparison & Operational Matrix

The following matrix summarizes the technical attributes, complexity, participant actors, and execution domains of all seven core algorithms implemented across the DocSafe system:

Algo #	Name & Function	Execution Domain	Primary Actors	Time Complexity	Gas Profile
Algo 1	Evidence Ingestion & Consent	Ethereum Smart Contract	Provider, Recipient	O(1) constant	~85k - 120k gas
Algo 2	Force Update & Challenge Flow	Ethereum Smart Contract	Provider, Recipient, Validators	O(1) per vote	~45k - 95k gas
Algo 3	MetaMask Signing & IPFS Binding	Client + IPFS Gateway	Provider, MetaMask, IPFS	O(N) data upload	~55k gas anchor
Algo 4	Dual-Digest Integrity Verification	Client WebCrypto + Express	Browser Client, API Server	O(N) streaming hash	Off-chain ($0 gas)
Algo 5	Rejection & Lineage Resubmission	Ethereum Smart Contract	Recipient, Validator, Provider	O(1) constant	~35k - 65k gas
Algo 6	Dynamic RBAC Viewer Delegation	Ethereum Smart Contract	Recipient, 3rd-Party Viewers	O(1) lookup/grant	~28k gas per grant
Algo 7	Chain-of-Custody Timeline Aggregation	Client Viem RPC + SQLite	Auditors, Legal Counsel, Court	O(L log L) sort	Off-chain ($0 gas)

10. Academic References & Project Citations

The algorithmic formulations implemented in this project are substantiated by the research literature referenced in the DocSafe research paper drafts:

[1] S. Bonomi, M. Casini, and C. Ciccotelli, "B-CoC: A blockchain-based chain of custody for evidences management in digital forensics," arXiv:1807.10359, 2018.

[2] S. H. Gopalan, S. A. Suba, C. Ashmithashree, A. Gayathri, and V. J. Andrews, "Digital forensics using blockchain," Int. J. Recent Technol. Eng., vol. 8, pp. 182–184, 2019.

[3] J. Bou Abdo, R. El Sibai, and J. Demerjian, "Permissionless proof-of-reputation-X: A hybrid reputation-based consensus algorithm," Trans. Emerg. Telecommun. Technol., vol. 32, p. e4148, 2021.

[4] T. Varshney, N. Sharma, I. Kaushik, and B. Bhushan, "Authentication and encryption based security services in blockchain technology," in Proc. IEEE ICCCIS, 2019, pp. 63–68.

[5] National Institute of Standards and Technology (NIST), "Blockchain Technology Overview," NIST IR 8202, Oct. 2018. doi: 10.6028/NIST.IR.8202.

[6] E. Androulaki et al., "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in Proc. Thirteenth EuroSys Conf. (EuroSys '18), 2018, Art. 30.